

이삼이오_방캅스_3주차

3주차 — 정적 분석 모듈 구현 (~ 월요일 21:00)

구조 설계

- Python 모듈 구조 설계 — `static_analyzer/` 패키지, 오케스트레이터가 호출할 인터페이스 함수 `analyze_static(apk_path) -> dict` 하나로 통일

디컴파일 연동

- `subprocess.run()` 으로 `apktool d` 와 `jadx` CLI를 호출하는 wrapper 함수 작성 — 타임아웃 설정, 디컴파일 실패 시 예외 처리 포함

Manifest 파싱

- `AndroidManifest.xml` 파싱 — `androguard` (권한/컴포넌트 추출 API 내장, 빠르게 개발 가능) 또는 `xml.etree.ElementTree` (직접 파싱, 의존성 적음) 중 택1. 팀 역량과 일정 고려해 `androguard` 우선 검토 권장
- 위험 권한 목록 및 가중치 테이블 정의 — 예: `BIND_ACCESSIBILITY_SERVICE` (고위험), `SYSTEM_ALERT_WINDOW` (고위험), `READ_SMS / RECEIVE_SMS` (고위험), `REQUEST_INSTALL_PACKAGES` (중위험) 등을 표로 정리
- 컴포넌트(액티비티/서비스/브로드캐스트 리시버) 및 인텐트 필터 추출 — `exported="true"` 인 컴포넌트는 외부 공격면이 될 수 있으므로 위험 요소로 함께 표시

점수화 및 출력

- 위험도 점수 계산 로직 구현 — 1차는 권한별 가중치 합산 방식으로, 점수를 LOW/MEDIUM/HIGH 등급으로도 매핑
- `static_report.json` 스키마 확정 및 출력 함수 구현 (권한 목록, 위험 권한 하이라이트, 점수/등급, 컴포넌트 목록)

검증

- 정상 앱 2~3개 vs 공개 악성 샘플 2~3개로 점수 차이가 실제로 나는지 확인 (간단한 pytest 유닛 테스트로 회귀 방지)
- 1차 발표 준비:** 기획안, 시스템 구조도, 정적 분석 스크립트 실행 데모 (터미널 캡처 or 라이브 시연)

4명 역할 분배

A — 구조 설계

- Python 모듈 구조 설계 — `static_analyzer/` 패키지, 오케스트레이터가 호출할 인터페이스 함수 `analyze_static(apk_path) -> dict` 확정
- 이 인터페이스(입출력 형식)를 제일 먼저 정해서 B/C/D에게 공유해야 나머지가 그 형식에 맞춰 개발 가능

B — 디컴파일 연동

- `subprocess.run()` 으로 `apktool d`, `jadx` CLI 호출하는 wrapper 함수 작성
- 타임아웃 설정, 디컴파일 실패 시 예외 처리

C — Manifest 파싱

- `androguard` 또는 `ElementTree`로 `AndroidManifest.xml` 파싱
- 위험 권한 목록 + 가중치 테이블 정리
- 컴포넌트(액티비티/서비스/리시버) 및 인텐트 필터 추출, `exported="true"` 체크

D — 점수화 및 출력

- 위험도 점수 계산 로직 (권한 가중치 합산 → LOW/MEDIUM/HIGH)
- `static_report.json` 스키마 확정 및 출력 함수 구현
- C가 뽑은 파싱 결과를 입력으로 받으므로, C와 필드명만 미리 맞춰두면 됨